Sistema Nacional

Bitácora de cambios - Entrega 3

7 cambios registrados. El más reciente figura primero.

---

7. Blueprint de Render para el despliegue

21 de octubre de 2026

Se agregó render.yaml en la raíz del repositorio para crear en Render, con un solo paso, el servicio web del Sistema Nacional y su base PostgreSQL gestionada.

Qué se hizo

- render.yaml (raíz del repo): servicio web docker en plan free, construido desde sistema_nacional/Dockerfile con contexto sistema_nacional (el último stage, runtime), chequeo de salud en /health y build filter para que solo redespliegue cuando cambia sistema_nacional/.
- Variables de entorno: DATABASE_URL se toma de la base gestionada (sistema-nacional-db); JWT_SECRET lo genera Render con un valor aleatorio; ADMIN_EMAIL fijo; ADMIN_PASSWORD se pide al crear el Blueprint y no queda en el repositorio.
- Base de datos PostgreSQL gestionada (plan free, base y usuario sistema_nacional). Sin ?schema= en la URL el sistema usa el schema public.

Archivos

- render.yaml (nuevo)

Verificación

- El YAML se validó con un parser; los campos se contrastaron con la documentación de Blueprints de Render (el campo dockerTarget no existe, por eso el Dockerfile deja runtime como último stage).
- La imagen que construirá Render ya se había probado localmente (migración, seed, /health y login) con el mismo contexto y Dockerfile.
- No se verificó el despliegue real: requiere la cuenta de Render y que la rama esté subida a GitHub.

A tener en cuenta

- Plan free: el servicio se duerme tras 15 minutos sin tráfico y la primera petición tarda cerca de un minuto en despertarlo; tener en cuenta para llamadas desde Bonita o el backend.
- Plan free: la base expira a los 30 días de creada (con 14 días de gracia antes de borrarse) y no tiene backups. Si se crea el 22/10 vence cerca del 21/11, antes del coloquio del 26/11. Como el seed y las migraciones son reproducibles, se puede recrear la base cerca de la entrega final.
- Después del primer despliegue hay que guardar la URL pública y la contraseña ADMIN para configurar el cliente en el backend local.

---

6. Dockerfile de producción y compose corregido

21 de octubre de 2026

El Dockerfile pasó a tener dos targets: uno de producción (el que usará Render) y uno de desarrollo (el que usa docker compose). Se corrigió el compose para que funcione de punta a punta sin archivos .env.

Qué se hizo

- Dockerfile multi-stage. Target runtime (último, se construye por defecto): compila en una etapa aparte, instala solo dependencias de producción, corre como usuario node y al arrancar ejecuta prisma migrate deploy, el seed (idempotente) y node dist/server.js. Render inyecta PORT. Target dev: arranque con tsx watch, como lo dejó Pedro.
- .dockerignore nuevo: ya no entran a la imagen node_modules, dist, .env, el cliente generado, docs ni PDFs.

- El seed se movió de prisma/seed.ts a src/seed.ts para que se compile a dist/seed.js y corra en producción sin tsx. npm run db:seed sigue funcionando igual.
- prisma pasó de devDependencies a dependencies: hace falta en producción para migrate deploy.
- docker-compose.yml: el servicio usa target dev; DATABASE_URL apunta al host postgres:5432 de la red de compose (antes apuntaba a localhost, que dentro del contenedor no es la base); JWT_SECRET, ADMIN_EMAIL y ADMIN_PASSWORD tienen valores de desarrollo por defecto y se pueden pisar con SN_JWT_SECRET, SN_ADMIN_EMAIL y SN_ADMIN_PASSWORD. Ya no depende de un .env ni de la opción links. El watch ignora node_modules, dist y el cliente generado.
- entrypoint.sh (desarrollo): set -e y exec para que el proceso reciba las señales.
- .env.example: el puerto de la base pasa a 5440, que es el que publica el compose.

Archivos

- sistema_nacional/Dockerfile
- sistema_nacional/.dockerignore (nuevo)
- sistema_nacional/entrypoint.sh
- sistema_nacional/src/seed.ts (movido desde prisma/seed.ts)
- sistema_nacional/package.json y package-lock.json
- sistema_nacional/.env.example
- docker-compose.yml

Verificación

- Imagen runtime construida y ejecutada contra el Postgres del compose con un schema descartable: aplicó las tres migraciones, corrió el seed, /health respondió ok, el login con el ADMIN devolvió token y el proceso corre como usuario node. El schema de prueba se borró.
- docker compose up --build sistema_nacional (target dev) desde un clon sin .env: migró, sembró, arrancó con tsx watch en el puerto 3001 y el login respondió.
- npm run typecheck y npm test (8 tests) siguen pasando.

A tener en cuenta

- La imagen de producción pesa unos 530 MB, sobre todo por el CLI de Prisma. Es aceptable para Render, se puede reducir más adelante.
- docker-compose.yml es un archivo compartido que modificó Pedro: avisarle del cambio de DATABASE_URL, target y variables.
- El README y backend/.env.example de la raíz siguen indicando el puerto 5432 para Postgres; el compose publica 5440.
- Pendiente: Swagger, preparar el deploy en Render (variables, base gestionada) y actualizar la guía.

---

5. Liberación y cierre de compromisos

21 de octubre de 2026

Nuevo endpoint que devuelve al disponible los recursos bloqueados cuando una ONG termina su actividad o cuando se cierra la emergencia. Es idempotente y seguro ante llamadas simultáneas.

Qué se hizo

- POST /compromisos/liberar (ADMIN): recibe {emergenciaId} y opcionalmente ongId (id o externalId) y/o loteId. Libera los compromisos ACTIVOS que coincidan: resta lo bloqueado y los marca LIBERADO con la fecha.
- Dos usos con el mismo endpoint: una ONG termina su actividad (emergenciaId + ongId) y cierre del proyecto (solo emergenciaId, libera a todas las ONGs).
- Idempotente: repetir la llamada no resta de nuevo y responde 200 con liberados: 0 y yaLiberados: n. Si no hay ningún compromiso para ese criterio responde 404.
- Concurrencia: toma los mismos locks FOR UPDATE que el bloqueo, en el mismo orden (filas de inventario por id), y recién con el lock lee los compromisos ACTIVOS. Así dos liberaciones simultáneas del mismo compromiso restan una sola vez y no hay deadlock contra una adjudicación en curso.

- Aclaración de comportamiento: un compromiso liberado solo bloquea una re-adjudicación con la misma clave (misma emergencia, lote y ONG), que es un reintento tardío. Una emergencia nueva que pide los mismos recursos a la misma ONG es otra clave y se bloquea con normalidad.

Archivos

- src/compromisos.ts
- src/compromisos.test.ts

Verificación

- Tests nuevos (npm test, 8 en total pasan): liberar devuelve el stock; repetirla no vuelve a restar; una emergencia nueva puede volver a pedir los mismos 10 recursos a la misma ONG; el reintento tardío de la emergencia liberada da 409; 6 liberaciones simultáneas restan una sola vez; liberar por ONG y lote solo toca lo pedido; sin compromisos da 404.
- Prueba de que el test de concurrencia es real: sacando FOR UPDATE de la liberación falla de forma consistente (3 corridas).
- Probado con curl: liberación 200, repetición 200 con liberados 0, 404, 400 cuerpo inválido y 401 sin token; la disponibilidad global vuelve a su valor.

A tener en cuenta

- Con esto quedan implementados los cinco servicios mínimos de la API (login y onboarding, catálogo, validación, bloqueo y liberación).
- Pendiente: Swagger, Dockerfile de producción, despliegue en Render y el cliente del Sistema Nacional en el backend local.

---

4. Bloqueo transaccional de recursos con locking

21 de octubre de 2026

Nuevo endpoint para comprometer recursos de una ONG al adjudicar una oferta. Es todo o nada, idempotente y seguro ante adjudicaciones simultáneas.

Qué se hizo

- POST /compromisos (ADMIN): recibe {emergenciaId, loteId, ongId (id o externalId), items [{tipoRecurso, cantidad}]}, suma la cantidad a OngRecurso.bloqueado y crea el Compromiso con sus ítems.
- Locking: dentro de una transacción, SELECT ... FOR UPDATE sobre las filas de inventario de la ONG, ordenadas por id para evitar deadlocks. Dos adjudicaciones simultáneas sobre el mismo stock se serializan y la segunda ve lo ya bloqueado.
- Todo o nada: si algún ítem no alcanza, no se bloquea ninguno. La respuesta es 409 con detalle por ítem (solicitado, disponible, resultado, motivo), no un error genérico.
- Al adjudicar se exige ONG habilitada, certificación vigente del tipo y cantidad disponible. No se filtra por nivel de riesgo: una oferta de APOYO también se puede adjudicar.
- Idempotente por (emergenciaId, loteId, ongId): repetir la llamada devuelve el compromiso existente (200, repetido: true). La comprobación se hace con el lock tomado, así que los reintentos simultáneos tampoco duplican. Si el compromiso ya fue liberado responde 409.
- GET /compromisos (ADMIN): lista compromisos, con filtros emergenciaId y estado.
- src/db.ts exporta tabla(nombre): el SQL crudo necesita el schema calificado porque el adapter no ajusta el search_path (sin esto el FOR UPDATE falla con 'relation does not exist').

Archivos

- src/compromisos.ts (nuevo)
- src/compromisos.test.ts (nuevo)
- src/db.ts
- src/app.ts

Verificación

- Tests de concurrencia (npm test, necesitan la base): dos adjudicaciones de 6 sobre 10 -> una entra y una da 409 informando disponible 4; 12 adjudicaciones de 3 en paralelo sobre 10 -> exactamente 3 entran y bloqueado queda en 9; 5 reintentos simultáneos del mismo compromiso -> se aplica una sola vez; ítem insuficiente -> no se bloquea ninguno.
- Prueba de que los tests son reales: sacando FOR UPDATE fallan 3 de 4 de forma consistente (3 corridas).
- Probado con curl: bloqueo 201, repetición 200, 409 con detalle por falta de stock o de certificación, 404 ONG inexistente, 400 cuerpo inválido; la disponibilidad global refleja lo bloqueado.

A tener en cuenta

- Los tests crean y borran su propia ONG de prueba, pero usan la base del .env.
- Un compromiso ya liberado con la misma clave devuelve 409; si hace falta re-adjudicar el mismo lote a la misma ONG habría que definir el comportamiento.
- Pendiente: liberación y cierre (próximo paso).

---

3. Validación de ofertas por niveles de competencia

21 de octubre de 2026

Nuevo endpoint que evalúa ofertas y devuelve un resultado estructurado por oferta, sin rechazos binarios. Solo lee: no bloquea ni descuenta inventario.

Qué se hizo

- POST /validaciones (ADMIN): recibe hasta 500 ofertas [{ofertaId, ongId (id o externalId), tipoRecurso, cantidad, nivelRiesgo}] y devuelve un resultado por oferta.
- Regla de negocio: cada oferta se evalúa por separado contra el disponible (total - bloqueado). Una ONG con 10 puede ofertar muchas veces 10 o menos; el inventario se descuenta recién al adjudicar.
- Resultados: COMPLETA (categoría, riesgo y cantidad ok), PARCIAL (cantidad válida menor a la pedida), APOYO (riesgo del lote mayor al que habilitan sus certificaciones; sirve como apoyo secundario) y SIN_CAPACIDAD (ONG no habilitada o inexistente, sin certificación vigente, sin disponibilidad o tipo desconocido).
- Cada resultado incluye cantidadValida, disponible, nivelRiesgoMax y un motivo. Las certificaciones vencidas no cuentan.
- La lógica vive en src/competencia.ts, sin acceso a la base, para poder testearla sola. Se agregó npm test.

Archivos

- src/competencia.ts (nuevo)
- src/competencia.test.ts (nuevo)
- src/validacion.ts (nuevo)
- src/app.ts
- package.json
- tsconfig.json (excluye los tests del build)

Verificación

- npm test: pasa (completa, parcial, apoyo, sin capacidad, mayor nivel entre varias certificaciones).
- Probado con ocho ofertas reales: COMPLETA, PARCIAL, APOYO, certificación vencida, tipo sin certificación, ONG inexistente y tipo desconocido.
- Cuerpo inválido da 400; sin token da 401.

A tener en cuenta

- El resultado es una foto del momento: el bloqueo al adjudicar vuelve a comprobar la disponibilidad.
- Dos ofertas de 10 sobre un inventario de 10 pasan ambas la validación. Si se adjudican las dos, la segunda debe fallar al bloquear con un 409 estructurado (pendiente de implementar).

---

2. Identificación por externalId y CUIT opcional

21 de octubre de 2026

Se adaptó el modelo para integrarlo con la app local: la ONG se identifica con el id de su usuario local y el CUIT dejó de ser obligatorio.

Qué se hizo

- Ong.externalId (único): id del usuario ONG en la app local.
- Ong.cuit pasa a ser opcional (sigue siendo único si se informa).
- POST /ongs (ADMIN): alta o actualización idempotente por externalId, con certificaciones y recursos opcionales. No crea usuario. Una ONG nueva queda PENDIENTE salvo que se indique estado.
- Todas las rutas /ongs/:id aceptan el id interno o el externalId (que no puede ser solo numérico).
- /register: el CUIT pasa a ser opcional.

Archivos

- prisma/schema.prisma
- prisma/migrations/20261021120000_ong_external_id/migration.sql
- src/ongs.ts

Verificación

- Alta sin CUIT con certificaciones y recursos; repetir el alta actualiza sin duplicar (200).
- externalId numérico rechazado (400); externalId inexistente da 404; dos ONGs sin CUIT conviven.
- prisma migrate diff: sin diferencias entre schema y base.

A tener en cuenta

- Si el alta trae certificaciones inválidas devuelve 400, pero la ONG ya quedó creada (la respuesta incluye su ongId). Reintentar con datos corregidos funciona.

---

1. Registro de ONGs y catálogo

21 de octubre de 2026

Se agregaron el onboarding de ONGs, la gestión por el ADMIN y los endpoints de consulta del catálogo. (Entrada registrada a posteriori.)

Qué se hizo

- POST /register (público): crea la ONG en estado PENDIENTE y su usuario en una sola transacción. Valida email, contraseña (8 o más) y CUIT si viene.
- PATCH /ongs/:id/estado (ADMIN): habilita, suspende o vuelve a PENDIENTE.
- PUT /ongs/:id/certificaciones y PUT /ongs/:id/recursos (ADMIN): cargan certificaciones (con vencimiento) y total por tipo de recurso. No tocan lo bloqueado.
- GET /certificaciones, /tipos-recurso, /ongs (habilitadas), /ongs/me, /ongs/:id y /recursos/disponibilidad (cualquier usuario autenticado; una ONG solo ve la suya).
- Manejador de errores: JSON roto da 400 y los errores internos dan 500 sin detalles.

Archivos

- src/ongs.ts (nuevo)
- src/app.ts

Verificación

- Probado con curl contra la API real: registro (400, 201, 409), permisos (401, 403), certificaciones vencidas, habilitación, disponibilidad global y CHECK de recursos bloqueados (409).
- No probado: ONG que pide el perfil de otra (403) y CUIT repetido por un ADMIN.

A tener en cuenta

- La base local de Postgres se usa en el puerto 5440 (el compose de Pedro cambió el 5432).